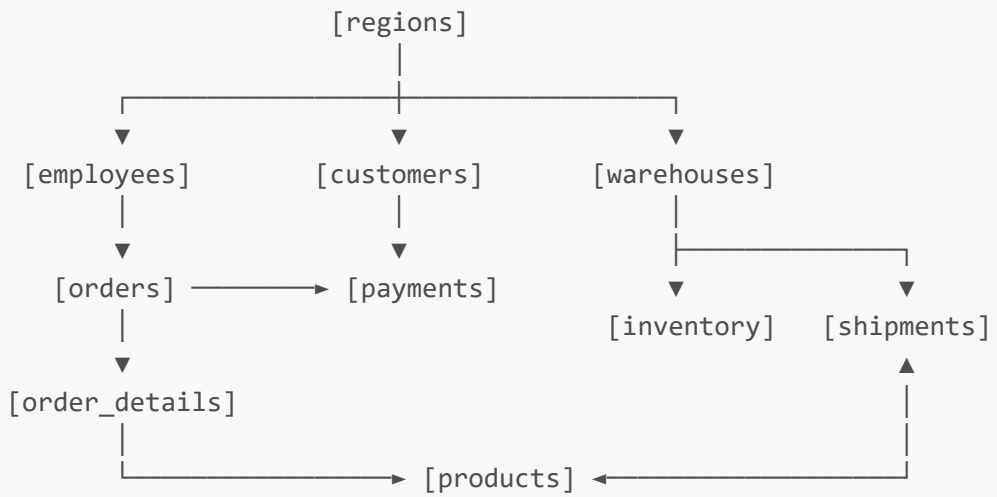
DecisionIQ Master Handbook

CHAPTER 2: Data Architecture & Database Design

2.1 The Star/Snowflake Schema Strategy

An enterprise data warehouse is the foundation of any analytics product. If your database design is poorly structured, your analytical queries will be slow, your machine learning features will contain errors, and your dashboard widgets will load sluggishly.

To design the DecisionIQ data warehouse, we use a hybrid **Star/Snowflake Schema** architecture.



2.2.1 Why Snowflake over Flat Tables?

In a flat table design (often seen in basic Kaggle data projects), all data is stored in a single large CSV. While easy to read in Pandas, a single flat table is a production nightmare:

1. **Massive Redundancy:** If a customer places 100 orders, their company name, contact email, and address are duplicated 100 times, wasting storage.
2. **Update Anomalies:** If a customer changes their contact email, you must scan and update thousands of transaction rows. If you miss one row, your database becomes inconsistent.
3. **No Referential Integrity:** There is nothing preventing an order from referencing a customer ID that does not exist.

DecisionIQ normalizes lookup attributes (dimensions) like `regions` and `products` into their own tables, linking them to transactions (facts) like `orders` and `shipments` via foreign keys. This guarantees that each entity exists in exactly one place (Single Source of Truth).

2.2 Table Schemas & Constraints

The DecisionIQ schema consists of 14 tables, designed with strict constraints to preserve data quality at the storage layer.

1. regions (Dimension)

Represents geographic operations territories.

- `region_id` (VARCHAR, PK): Unique code (e.g., `REG01`).
- `region_name` (VARCHAR, NOT NULL): Description (e.g., `North America`).
- `country` (VARCHAR, NOT NULL): Base country.
- `manager_name` (VARCHAR, NOT NULL): Regional director name.

2. employees (Dimension)

A hierarchical dimension containing staffing lines.

- `employee_id` (VARCHAR, PK): Employee identifier.
- `first_name` (VARCHAR, NOT NULL), `last_name` (VARCHAR, NOT NULL).
- `email` (VARCHAR, UNIQUE NOT NULL): Corporate email.
- `role` (VARCHAR, NOT NULL): Job title (e.g., `Sales Representative`).
- `department` (VARCHAR, NOT NULL): Department division.
- `region_id` (VARCHAR, FK): References `regions(region_id)` (Null for global execs).
- `reports_to` (VARCHAR, FK): References `employees(employee_id)` (Null for the CEO).

3. customers (Dimension)

Contains B2B client details.

- `customer_id` (VARCHAR, PK): Client account code.
- `company_name` (VARCHAR, NOT NULL).
- `contact_email` (VARCHAR, UNIQUE NOT NULL).
- `customer_segment` (VARCHAR, NOT NULL): Segment tier (`SMB`, `Enterprise`, `Strategic`).
- `region_id` (VARCHAR, FK): References `regions(region_id)`.
- `status` (VARCHAR, NOT NULL): Health flag (`Active`, `Churned`, `At-Risk`).
- `acquisition_date` (DATE, NOT NULL).

4. products (Dimension)

Contains catalog hardware and software listings.

- `product_id` (VARCHAR, PK): Product code.
- `sku` (VARCHAR, UNIQUE NOT NULL): Stock keeping identifier.
- `category` (VARCHAR, NOT NULL): Category division (`Software`, `Hardware`, `Support`, `Services`).
- `unit_price` (DECIMAL, NOT NULL): CHECK `unit_price >= 0`.
- `unit_cost` (DECIMAL, NOT NULL): CHECK `unit_cost >= 0`.
- `status` (VARCHAR, NOT NULL): Lifecycle flag (`Active`, `Discontinued`).

5. warehouses (Dimension)

Physical distribution fulfillment nodes.

- `warehouse_id` (VARCHAR, PK): Facility identifier.
- `warehouse_name` (VARCHAR, NOT NULL).
- `location` (VARCHAR, NOT NULL).
- `region_id` (VARCHAR, FK): References `regions(region_id)`.
- `capacity_sqft` (INTEGER, NOT NULL).
- `operating_cost_monthly` (DECIMAL, NOT NULL): CHECK `>= 0`.

6. inventory (Fact/Bridge)

Tracks physical stock holdings in warehouses.

- `inventory_id` (VARCHAR, PK): Unique code.
- `warehouse_id` (VARCHAR, FK): References `warehouses(warehouse_id)`.
- `product_id` (VARCHAR, FK): References `products(product_id)`.
- `quantity_on_hand` (INTEGER, NOT NULL): CHECK `>= 0`.
- `reorder_point` (INTEGER, NOT NULL): CHECK `>= 0`.
- `last_stock_count_date` (DATE, NOT NULL).
- *Constraint:* `UNIQUE(warehouse_id, product_id)` prevents duplicate lines.

7. suppliers (Dimension)

Fulfillment component providers.

- `supplier_id` (VARCHAR, PK): Supplier code.
- `supplier_name` (VARCHAR, NOT NULL).
- `lead_time_days` (INTEGER, NOT NULL): CHECK `>= 0`.
- `reliability_score` (DECIMAL, NOT NULL): CHECK `>= 0.0 AND <= 1.0`.

8. orders (Fact)

Transaction headers.

- `order_id` (VARCHAR, PK): Order invoice number.
- `customer_id` (VARCHAR, FK): References `customers(customer_id)`.
- `order_date` (DATE, NOT NULL).
- `status` (VARCHAR, NOT NULL): Status (`Completed`, `Shipped`, `Processing`, `Cancelled`, `Refunded`).
- `sales_representative_id` (VARCHAR, FK): References `employees(employee_id)`.
- `total_amount` (DECIMAL, NOT NULL): CHECK `>= 0`.

9. order_details (Fact)

Granular order line items.

- `order_detail_id` (VARCHAR, PK).
- `order_id` (VARCHAR, FK): References `orders(order_id)`.
- `product_id` (VARCHAR, FK): References `products(product_id)`.
- `quantity` (INTEGER, NOT NULL): CHECK `quantity > 0`.

- `unit_price` (DECIMAL, NOT NULL): CHECK `>= 0`.
- `discount_amount` (DECIMAL, NOT NULL): DEFAULT 0.0, CHECK `>= 0`.
- `subtotal` (DECIMAL, NOT NULL): CHECK `>= 0`.

10. payments (Fact)

Monetary transaction settlements.

- `payment_id` (VARCHAR, PK).
- `order_id` (VARCHAR, FK): References `orders(order_id)`.
- `payment_date` (DATE, NOT NULL).
- `payment_method` (VARCHAR, NOT NULL): Method (`Credit Card`, `ACH`, `Wire`, `PayPal`).
- `payment_status` (VARCHAR, NOT NULL): Status (`Success`, `Failed`, `Refunded`).
- `amount` (DECIMAL, NOT NULL): CHECK `>= 0`.

11. shipments (Fact)

Tracks delivery fulfillment of physical hardware.

- `shipment_id` (VARCHAR, PK).
- `order_id` (VARCHAR, FK): References `orders(order_id)`.
- `warehouse_id` (VARCHAR, FK): References `warehouses(warehouse_id)`.
- `supplier_id` (VARCHAR, FK): References `suppliers(supplier_id)`.
- `carrier` (VARCHAR, NOT NULL): Carrier (`FedEx`, `UPS`, `DHL`, `Freight`).
- `shipment_date` (DATE).
- `estimated_delivery_date` (DATE, NOT NULL).
- `actual_delivery_date` (DATE).
- `delivery_status` (VARCHAR, NOT NULL): Status (`On Time`, `Late`, `In Transit`, `Returned`).

12. marketing_campaigns (Dimension)

Tracks ad campaign metrics.

- `campaign_id` (VARCHAR, PK).
- `campaign_name` (VARCHAR, NOT NULL).
- `channel` (VARCHAR, NOT NULL): Channel (`Social Media`, `PPC`, `Email`, `SEO`, `Event`).
- `start_date` (DATE, NOT NULL), `end_date` (DATE, NOT NULL).
- `budget` (DECIMAL, NOT NULL): CHECK `>= 0`.
- `clicks` (INTEGER, NOT NULL): CHECK `>= 0`.
- `impressions` (INTEGER, NOT NULL): CHECK `>= 0`.
- `conversions` (INTEGER, NOT NULL): CHECK `>= 0`.

13. customer_support (Fact)

Support ticket lifecycle.

- `ticket_id` (VARCHAR, PK).
- `customer_id` (VARCHAR, FK): References `customers(customer_id)`.
- `created_at` (TIMESTAMP, NOT NULL), `closed_at` (TIMESTAMP).

- `status` (VARCHAR, NOT NULL): Status (`Open`, `In Progress`, `Resolved`, `Escalated`).
- `priority` (VARCHAR, NOT NULL): Severity (`Low`, `Medium`, `High`, `Critical`).
- `category` (VARCHAR, NOT NULL): Category (`Billing`, `Technical`, `Product`, `Logistics`).
- `satisfaction_score` (INTEGER): CHECK `>= 1 AND <= 5`.
- `agent_id` (VARCHAR, FK): References `employees(employee_id)`.

14. daily_business_metrics (Fact)

Tracks daily high-level marketing aggregates.

- `metric_date` (DATE, PK).
- `active_users` (INTEGER, NOT NULL): CHECK `>= 0`.
- `website_visitors` (INTEGER, NOT NULL): CHECK `>= 0`.
- `leads_generated` (INTEGER, NOT NULL): CHECK `>= 0`.
- `cac` (DECIMAL, NOT NULL): CHECK `>= 0`.
- `nps` (INTEGER): CHECK `>= -100 AND <= 100`.

2.3 SQLite vs. PostgreSQL: Dual Database Design

DecisionIQ is engineered to work seamlessly with two different database backends using the exact same schema code.

2.3.1 Local Development Mode (SQLite)

- **Storage:** Stored as a single local file (`decision_iq.db`).
- **Role:** Ideal for local developers, testing scripts, and running dashboards immediately without configuring database server credentials.
- **Enforcing Foreign Keys:** SQLite has foreign key support disabled by default. We configure our SQLAlchemy database connector to automatically issue a connection listener callback:

```
@event.listen(engine, "connect")
def set_sqlite_pragma(dbapi_connection, connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()
```

2.3.2 Production Mode (PostgreSQL)

- **Storage:** Runs on a dedicated server (accessible via port `5432`).
- **Role:** Ideal for production analytics, multiple database readers, and large volumes of data.
- **Activation:** Activated automatically by setting the `DATABASE_URL` environment variable. Our ORM layer detects this and swaps out SQLite dialect syntax for PostgreSQL.

2.4 Indexing for High-Performance Queries

When a user opens the CEO Control Tower dashboard, the backend must join multiple large transactional tables to render charts. Without optimization, the database must perform a **Full Table Scan**, reading every single row from disk ($O(N)$ complexity).

To prevent lag, we create B-Tree indexes on frequently queried foreign key columns:

```
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
CREATE INDEX idx_orders_order_date ON orders(order_date);
CREATE INDEX idx_order_details_order_id ON order_details(order_id);
CREATE INDEX idx_payments_order_id ON payments(order_id);
CREATE INDEX idx_shipments_order_id ON shipments(order_id);
```

With these indexes, searching for a specific order drops from $O(N)$ to $O(\log N)$, ensuring sub-second response times even as the data warehouse scales to millions of records.